

Funzioni/Variabili Esterne e Statiche

Lezione 13

Sistemi Operativi - II Modulo
(Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Variabili esterne e statiche
- Funzioni statiche
- Ricorsione
- Puntatori a funzione
- gdb (*'disinfestatore'*)



Visibilità (*scoping*) delle variabili

- In C, una variabile locale (o anche automatica) è visibile solo nel **blocco** in cui è dichiarata
- Una variabile globale è visibile dal punto di dichiarazione fino alla fine del *file sorgente*

```
int  myvar;  
  
void fun1() {  
    if (myvar == 4) { ... }  
  
void main() {  
    myvar = 4;  
    fun1();  
}
```



Variabili esterne

- **Dichiarazione** di una variabile: affermo *tipo* e *dimensione* della var.
- **Definizione**: viene *allocata* anche la memoria per la var.
- Per usare una variabile che non è visibile (ad es. è definita in un altro file) la si dichiara con la clausola `extern`

```
extern  int    myvar;  
extern  char   str[];
```

Variabili esterne e visibilità delle funzioni

```
void func1() {  
    extern int myvar;                // myvar non ancora definita, uso extern  
    void nestfunc1() {  
        printf("sono nestfunc1\n");  
    }  
    nestfunc1();  
    myvar = 1;  
}  
int myvar = 0;                      // definizione di myvar!  
void main(void) {  
    printf("myvar = %d\n", myvar);  
    func1();  
    printf("myvar = %d\n", myvar);  
}
```

Il codice stampa:

```
myvar = 0  
sono nestfunc1  
myvar = 1
```



Esercizio

- Aprite `nestedf.c`
- Spostare la dichiarazione di `myvar` da `func1()` ad un altro file (chiamatelo ad esempio `myv.c`)
- Ricompilate il tutto: il programma dovrebbe stampare come prima
 - Dovete compilare entrambi i file, ad es.: `gcc nestedf.c myv.c`

Variabili statiche

- Possono essere locali o esterne
- ***Locali:***
 - dichiarate all'interno di una funzione
 - sono **mantenute** tra una chiamata e l'altra della funzione
 - Costituiscono una ***memoria privata*** della funzione

Variabili statiche

```
void fun() {  
    static int myvar;  
  
    myvar++;  
    printf("myvar = %d\n", myvar);  
}
```

```
void main(void) {  
    fun();  
    fun();  
}
```

Le variabili `static` vengono automaticamente inizializzate (una volta)

//inizializzata automaticamente a 0

Il codice stampa:

```
myvar = 1  
myvar = 2
```


Variabili statiche esterne

- Sono variabili statiche dichiarate *al di fuori* di ogni funzione (in un dato file sorgente)
- Sono visibili dal punto di dichiarazione fino alla fine del *file sorgente*

Funzioni statiche

- Non visibili al di fuori del file sorgente in cui sono dichiarate
- La clausola `static` permette quindi di *variare la visibilità* di un oggetto, non solo la sua permanenza in memoria

Funzioni statiche

```
// file fstatic.c

void fun() {
    static int myvar;

    myvar++;
    printf("myvar = %d\n", myvar);
}

void main(void) {
    fun();
}
```

```
// file fun.c

static void fun() {
    printf("sono fun() static\n");
}
```

Compiliamo ed eseguiamo:

```
$> gcc fstatic.c fun.c
$> ./a.out
myvar = 1
```



Esercizio

- Modificate l'esempio precedente in modo che `main()` chiami `fun()` del file `fun.c`



Ricorsione

- In C è possibile scrivere funzioni *ricorsive*, cioè che chiamano se stesse
- ***Svantaggi***: in generale codice più lento e più consumo di memoria (perché?)
 - [Variabili locali/automatiche salvate nello stack.]
- ***Vantaggi***: codice più compatto e semplice da scrivere
- Esercizio [max 5 minuti!]: scrivere una funzione ricorsiva per il fattoriale di un intero (provatela con numeri minori di 20...)

Puntatori a funzione

- Il C permette puntatori a funzione, anche se una funzione NON è una variabile!

- Dichiarazione:

```
int    (* funptr) ();
```

//funptr è un puntatore ad una funzione che ritorna un intero

- Chiamata:

```
int a;
```

```
a = (* funptr) ();
```



Puntatori a funzione

- Possono essere usati per rendere il codice più *generico*
- Vediamo:
 - `funptr.c`
 - `qsort-generic.c`



Puntatori a funzione: altri esempi

- Una forma “primitiva” di programmazione ad oggetti usando variabili statiche e puntatori a funzione.
- Vediamo:
 - `oo.[ch]` e `oo-main.c`



Esercizio

- Aggiungere a `oo.[ch]` un 'metodo' che permetta di applicare una qualsiasi funzione da `double` a `double` agli elementi della struttura `myclass`. Il metodo ritorna i risultati in un array.
 - Passate la funzione come parametro (ovviamente come puntatore a funzione).
 - Il metodo deve chiamare SOLO funzioni passate come parametro (potete metterle tutte in un array).

[Per usare le funzioni matematiche dovete includere `<math.h>` e compilare aggiungendo `-lm`]



gdb

Materiale didattico per cortesia del prof. Emanuele Gabrielli



'Disinfestare' programmi?

Vera tarma/falena trovata in un relè del Mark II ad Harvard, 1947.

92

9/9

0800 Antam started
1000 " stopped - antam ✓
1300 (032) MP - MC ~~1.982647000~~
(033) PRO 2 2.130476415
concl 2.130676415

Relays 6-2 in 033 failed special speed test
in relay 10.000 test.

Relays changed
1700 Started Cosine Tape (Sine check)
1525 Started Multi-Adder Test.

1545



Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.
~~1630~~ Antam started.
1700 closed down.

Relay
2145
Relay 3370



SAPIENZA
UNIVERSITÀ DI ROMA

‘Disinfestare’ programmi?

- Nello sviluppo di codice è spesso utile far girare e fermare un programma a piacimento, vedere il contenuto delle variabili, quali funzioni sono chiamate, etc.
- Con gcc, c'è l'opzione `-g`

```
gcc -g miosorgente.c
```

(vedi man per tutte le opzioni di debug di gcc)

`gdb`

- `gdb` è il GNU Project Debugger
- Open source, portabile, linea di comando. Linguaggi supportati:
 - Ada
 - Assembly
 - C
 - C++
 - D
 - Fortran
 - Go
 - Objective-C
 - OpenCL
 - Modula-2
 - Pascal
 - Rust



gdb

```
/* Esempio per debug con gdb */  
#include<stdio.h>  
main()  
{  
    int a=0;  
    a++;  
    a=a+1;  
    a=a+2;  
    a=a+3;  
    a=a+4;  
    a=a+5;  
    printf("Il valore di a e':%d\n",a);  
    return a+1;  
}
```



`gdb`

```
$gcc -g esempio.c -o esempio
```

```
$gdb
```

```
(gdb)help
```

```
help
```

```
List of classes of commands:
```

```
aliases -- Aliases of other commands
```

```
breakpoints -- Making program stop at certain points
```

```
data -- Examining data
```

```
files -- Specifying and examining files
```

```
...
```

```
running -- Running the program
```

```
Type "help" followed by a class name for a list of commands in that class.
```

```
Type "help all" for the list of all commands.
```

```
Type "help" followed by command name for full documentation.
```

```
Type "apropos word" to search for commands related to "word".
```

```
Command name abbreviations are allowed if unambiguous.
```



`gdb`

```
(gdb)help files
```

```
Specifying and examining files.
```

```
List of commands:
```

```
...
```

```
cd -- Set working directory to DIR for debugger and program being debugged
```

```
core-file -- Use FILE as core dump for examining memory and registers
```

```
directory -- Add directory DIR to beginning of search path for source files
```

```
edit -- Edit specified file or function
```

```
file -- Use FILE as program to be debugged
```

```
...
```

```
generate-core-file -- Save a core file with the current state of the debugged process
```

```
list -- List specified function or line
```

```
pwd -- Print working directory
```

```
...
```

```
(gdb)
```



`gdb`

```
(gdb) file esempio
```

```
Reading symbols from esempio...done.
```

```
(gdb) help list
```

```
List specified function or line.
```

```
With no argument, lists ten more lines after or around previous listing.
```

```
"list -" lists the ten lines before a previous ten-line listing.
```

```
One argument specifies a line, and ten lines are listed around that line.
```

```
Two arguments with comma between specify starting and ending lines to list.
```

```
Lines can be specified in these ways:
```

```
    LINENUM, to list around that line in current file,
```

```
    FILE:LINENUM, to list around that line in that file,
```

```
    FUNCTION, to list around beginning of that function,
```

```
    FILE:FUNCTION, to distinguish among like-named static functions.
```

```
    *ADDRESS, to list around the line containing that address.
```

```
With two args if one is empty it stands for ten lines away from the other  
arg
```

gdb

```
(gdb) list 1,20
```

```
1      /* Esempio per debug con gdb */
2      #include<stdio.h>
3      main()
4      {
5          int a=0;
6          a++;
7          a=a+1;
8          a=a+2;
9          a=a+3;
10         a=a+4;
11         a=a+5;
12         printf("Il valore di a e':%d\n",a);
13         return a+1;
14     }
15
```

`gdb`

```
(gdb)help running
```

```
...
```

```
continue -- Continue program being debugged
```

```
jump -- Continue program being debugged at specified line or address
```

```
kill -- Kill execution of program being debugged
```

```
next -- Step program
```

```
run -- Start debugged program
```

```
...
```

```
(gdb) run
```

```
Starting program: /root/gdb/esempio
```

```
Il valore di a e':16
```

```
[Inferior 1 (process 18504) exited with code 021]
```

gdb: punti di fermo (*breakpoint*)

```
(gdb) help breakpoints
```

```
....
```

```
(gdb)break 8
```

```
Breakpoint 1 at 0x8048438: file esempio.c
```

```
(gdb)run
```

```
Starting program: /root/gdb/esempio
```

```
Breakpoint 1, main () at esempio.c:8
```

```
8 a=a+2;
```

```
(gdb)help data
```

```
...
```

```
display -- Print value of expression EXP each time the program stops
```

```
...
```

```
(gdb)display a
```

```
1: a = 2
```

```
(gdb) list 1,20
```

```
1      /* Esempio per debug con gdb */
```

```
2      #include<stdio.h>
```

```
3      main()
```

```
4      {
```

```
5          int a=0;
```

```
6          a=1;
```

```
7          a=a+1;
```

```
8          a=a+2;
```

```
9          a=a+3;
```

```
10         a=a+4;
```

```
11         a=a+5;
```

```
12         printf("Il valore di a e' :%d\n",a);
```

```
13         return a+1;
```

```
14     }
```

gdb: punti di fermo (*breakpoint*)

```
(gdb) help running
```

```
...
```

```
continue -- Continue program being debugged
```

```
jump -- Continue program being debugged at specified line or address
```

```
kill -- Kill execution of program being debugged
```

```
step -- Step program until it reaches a different source line
```

```
...
```

```
(gdb) step
```

```
9             a=a+3;
```

```
1: a = 4
```

```
(gdb) jump 11
```

```
Continuing at 0x8048447.
```

```
Il valore di a e':9
```

```
[Inferior 1 (process 18877) exited with
```

```
(gdb) list 1,20
```

```
1             /* Esempio per debug con gdb */
```

```
2             #include<stdio.h>
```

```
3             main()
```

```
4             {
```

```
5                 int a=0;
```

```
6                 a++;
```

```
7                 a=a+1;
```

```
8                 a=a+2;
```

```
9                 a=a+3;
```

```
10                a=a+4;
```

```
11                a=a+5;
```

```
12                printf("Il valore di a e':%d\n",a);
```

```
13                return a+1;
```

```
14            }
```

gdb: rimuovere le informazioni di debug

- `strip` rimuove tutte le info di debug da un file oggetto/eseguibile

```
strip miosorgente.o
```

```
$strip esempio
```

```
$gdb esempio
```

```
... .
```

```
Reading symbols from esempio...(no debugging symbols found)...done.
```

```
$
```

(vedi man per tutte le opzioni di strip)

Domande?

